

Product Variants Architecture: Color & Image Upload Analysis

Comprehensive technical evaluation of parent-child relationships, limitations, and actionable improvement recommendations.

Generated: September 2026 | Regal Horeca Catalog

ARCHITECTURE 01

Table 1: Color Section Drawbacks (Parent vs. Child Relation)

An analysis of how colors are defined on parent products vs. how they are connected to sellable child variant rows.

ISSUE	PARENT SIDE	CHILD (VARIANT) SIDE	ARCHITECTURAL DRAWBACK & CONSEQUENCE
Image Propagation	Admin uploads marketing images to colorVariants[].images.	Child stores own images[] / gallery and derives heroImage.	Images uploaded per color on the parent are never copied to child products during variant generation or save. Child variants end up falling back to the generic parent hero image unless re-uploaded per row.
Data Type Mismatch	Stored as structured array: { colorName, colorHex, images, isDefault }.	Stored as unindexed plain string: variationAttributes.color = "Red".	Loose string matching instead of relational IDs. Renaming or editing a color on the parent breaks the match on children, causing storefront swatches to fall back to a generic gray (#CCCCCC) circle.
UI Contradiction	Form text states: "Optional marketing swatches... sellable colors are set in Variants above."	Matrix generator requires fullFormColorNames from Color section.	The UI explicitly calls the Color section optional, but clicking "Generate Variants" with Color checked throws a hard validation error blocking creation until colors are added in that exact section.
Single Row Desync	Color list contains only ticked/custom colors from parent form.	Single row dropdown displays all 17 system predefined colors.	Admin can manually select "Green" in a variant row even if "Green" was never selected on the parent. The child is saved with "Green", but the parent has no hex, swatch, or marketing definition for it.
Deletion Cascading	Deleting a custom color removes it from parent.colorVariants.	Children continue referencing the deleted color string.	Parent color deletion does not validate or cascade to existing children. Child products become orphaned with a dangling color name that no longer exists in the parent palette.
Child Edit Lockout	Holds full permissions to edit swatches, color names, and images.	Color section is read-only; variationAttributes is stripped on direct child save.	If a child variant has an incorrect color assigned, it cannot be corrected from the child edit screen. The admin must navigate back to the parent product, find that row in the matrix, and re-save the parent.

Table 2: Image Upload Workflow Limitations & What Needs to Get Better

Identification of bottlenecks, data loss vulnerabilities, UI constraints, and actionable improvements for variant image management.

FEATURE AREA	CURRENT BEHAVIOR & DRAWBACK	WHAT NEEDS TO GET BETTER (ACTIONABLE SOLUTION)
Direct Child Edit (Data Loss)	Image upload inputs (Hero & Gallery) are active when editing a child product directly, but on save, heroImage and gallery are stripped by stripChildVariantOwnedFields(), silently discarding newly uploaded images.	Either allow direct child image updates by removing them from the strip list in PATCH /api/admin/products/children/[id], or disable the upload input in child view with a clear tooltip: "Managed via parent product".
Storefront Fallback (Dead Code)	When saving parent variants, empty variant rows automatically receive [parent.heroImage]. On the storefront (PDP), because variantImages.length > 0 is always true, the fallback to matchedColorVariant.images is never reached.	Store explicit empty arrays on variants that do not have dedicated photos, and update storefront PDP logic to check whether the image is genuinely variant-specific before bypassing parent color swatch galleries.
Batch Assignment	Each variant matrix row has an isolated 160px file input. If a product has 5 sizes across 4 colors (20 rows), admin must upload the red photo 5 separate times into 5 individual table cells.	Add bulk action buttons in the variant matrix header: "Apply this image to all variants of Color [X]" and "Copy row images to matching color group" to eliminate repetitive file uploads.
Image Management UI	Constrained 160px table cell with tiny 32x32px thumbnails. Images beyond 4 are truncated as "+N". No ability to reorder images, inspect previews, or choose which photo serves as the variant hero image.	Implement an "Edit Variant Images" popup modal with high-res previews, drag-and-drop reordering, and a "Set as Hero" badge (saving admin from deleting and re-uploading to change image order).
Upload State & Freezing	A single global isUploading boolean locks and freezes every single row upload button in the entire table simultaneously. No progress percentage or indicator showing which specific row is uploading.	Track upload state per row (e.g., uploadingRowIndex: Set<number>). Keep the rest of the form responsive while individual rows display a progress spinner and percentage indicator.
Parent Gallery Inheritance	When a child has even 1 specific image, resolveProduct() marks child.gallery as non-empty. Storefront stops inheriting parent gallery images, losing secondary assets like size charts, certificates, and lifestyle shots.	Adopt a combined gallery strategy on the storefront: display variant-specific angle shots first, followed by shared parent gallery photos (lifestyle, dimensions, certifications).
Storage Lifecycle (R2 Leak)	Deleting an image from a variant row or deleting a variant row entirely only removes the URL string from state. The actual file remains in Cloudflare R2 forever as an unreferenced orphan.	Implement a background cleanup job or trigger an R2 delete endpoint when variant photos are permanently deleted, preventing accumulating storage costs.
Validation & Optimization	Client allows up to 15MB (MAX_INPUT_SIZE), but server rejects files over 10MB (maxSize = 10MB). Furthermore, files under 1.5MB skip compression entirely, allowing massive 4000px images into the catalog.	Align client and server size limits to 10MB. Enforce dimension resizing (max 1600x1600px) on all uploads regardless of file size to keep PDP load times and print sheets fast.